Document 12

Canon Decision Log and Rationale

1. Purpose of This Document

This document records the major design decisions made during the creation of the Trinity Engine v1 canon package.

Its purpose is to:

- explain why key canon decisions were made
- record rejected alternatives
- support transparent review with the implementation engineer
- help distinguish intentional canon choices from accidental canon defects
- support structured dispute handling if implementation issues reveal ambiguity, inconsistency, or incompleteness

This document is included for transparency and governance support.

It does **not** override the governing canon documents.
If any conflict exists between this document and the governing canon package, the governing canon documents control implementation behavior.

---

2. How to Use This Document

This document should be used to:

- understand the rationale behind major canon choices
- review whether a disputed implementation issue is caused by:
    - an intentional canon decision
    - a canon defect
    - an implementation defect
    - a fixture defect
- support collaborative review if implementation reveals a genuine canon problem

This document must not be used as a substitute for the governing canon documents.

If a real canon defect is discovered, the fix must be made in the governing canon documents, not only here.

---

3. Decision Entry Format

Each decision entry contains:

- `decision_id`
- `title`
- `decision`
- `rationale`
- `rejected_alternatives`
- `affected_documents`
- `implementation_risk_prevented`
- `folded_back_status`

---

4. Decision Entries

Decision D01

decision_id: `D01`
title: Engine boundary starts after input normalization

decision:
Trinity Engine v1 receives normalized, validated input only. Raw birthplace resolution, geocoding, place disambiguation, and timezone determination are outside engine scope.

rationale:
This decision was made to keep the engine deterministic, narrow, and free from intake ambiguity. Place resolution and timezone determination are normalization concerns, not deterministic calculation core logic.

rejected_alternatives:

- engine accepts raw birthplace text
- engine resolves place names internally
- engine accepts ambiguous timezone inputs

affected_documents:

- Document 01
- Document 04
- Document 09

implementation_risk_prevented:

- mixed intake and calculation responsibilities
- location ambiguity leaking into engine behavior

- hidden normalization behavior affecting determinism

folded_back_status:
folded_back

Decision D02

decision_id: `D02`
title: No separate Trinity derived calculations category in v1

decision:
There is no separate supported output category called Trinity derived calculations in v1. All supported outputs must belong explicitly to Astrology, Human Design, or Gene Keys.

rationale:
This decision was made to prevent vague scope and unsupported derivation creep. A separate derived category would create a loophole for outputs with no clear domain ownership.

rejected_alternatives:

- keeping a vague fourth category
- allowing unsupported derived structures to appear without domain assignment

affected_documents:

- Document 01
- Document 02
- Document 09

implementation_risk_prevented:

- undefined scope expansion
- downstream output ambiguity
- engineer interpretation of what counts as a valid derived object

folded_back_status:
folded_back

Decision D03

decision_id: `D03`
title: Supported scope is defined explicitly, not by shared memory

decision:
The master calculation specification was treated as the source of truth for supported objects

and calculation foundations, unless the final canon package explicitly revised or narrowed that material. Informal references to “the same objects from the PoC canon” were rejected as an acceptable basis for implementation.

rationale:
This decision was made because prior shared understanding is not canon. An engineer cannot implement memory. Only explicit artifacts are acceptable.

rejected_alternatives:

- relying on prior chat memory
- relying on PoC assumptions without explicit restatement
- mixing prototype and final canon sources informally

affected_documents:

- Document 02
- Document 03
- Document 05
- Document 06
- Document 09

implementation_risk_prevented:

- silent scope drift
- dependence on undocumented founder memory
- implementation ambiguity created by informal references

folded_back_status:
folded_back

Decision D04

decision_id: `D04`
title: Human Design v1 includes structural BodyGraph outputs

decision:
Human Design v1 includes:

- Personality activations
- Design activations
- Channels
- Centers
- Definition
- Type
- Authority
- Profile

- Incarnation Cross

rationale:
This decision was made because a thinner Human Design scope would be structurally incomplete relative to how a credible BodyGraph is understood. Excluding Channels, Centers, and Definition would undermine the integrity of the Human Design layer.

rejected_alternatives:

- activations only
- excluding Channels
- excluding Centers
- excluding Definition while still claiming meaningful Human Design support

affected_documents:

- Document 02
- Document 05
- Document 06
- Document 07
- Document 09

implementation_risk_prevented:

- incomplete Human Design implementation
- mismatch between supported scope and expected BodyGraph structure
- later rework caused by artificially thin v1 scope

folded_back_status:
folded_back

Decision D05

decision_id: `D05`
title: Gene Keys remains identifier only in engine scope

decision:
Gene Keys v1 includes identifiers and Activation Sequence positions only. Shadow, gift, and essence text remain outside engine scope.

rationale:
This decision preserves the separation between deterministic calculation and content rendering. The engine should return structural identifiers, not semantic prose.

rejected_alternatives:

- generating Gene Keys prose in engine output
- embedding shadow, gift, and essence text in the calculation layer

affected_documents:

- Document 02
- Document 05
- Document 06
- Document 07
- Document 09

implementation_risk_prevented:

- contamination of engine with content logic
- version instability caused by wording changes
- mixing deterministic output with presentation layer concerns

folded_back_status:
folded_back

Decision D06

decision_id: `D06`
title: Engine is stateless and persistence is outside scope

decision:
Trinity Engine v1 is defined as a stateless calculation engine. It does not persist user profile data, calculation state, run history, or execution artifacts as part of its core responsibility.

rationale:
This decision was made to preserve deterministic purity, simplify testing, and prevent hidden state from influencing results.

rejected_alternatives:

- storing calculation state inside the engine
- embedding persistence responsibilities in the engine core
- allowing internal cached behavior to affect canonical output

affected_documents:

- Document 01
- Document 08
- Document 09

implementation_risk_prevented:

- hidden state drift
- harder reproducibility
- unclear responsibility boundaries between engine and infrastructure

folded_back_status:
folded_back

Decision D07

decision_id: `D07`
title: Strict numeric typing at the engine boundary

decision:
Latitude and longitude must be true numeric values. Numeric looking strings are rejected.

rationale:
This decision was made to preserve a strict normalized engine boundary and eliminate silent type coercion.

rejected_alternatives:

- accepting numeric strings
- silently coercing numeric strings into numbers

affected_documents:

- Document 04
- Document 09

implementation_risk_prevented:

- fake input contract
- hidden repair behavior
- inconsistent upstream formatting silently accepted by the engine

folded_back_status:
folded_back

Decision D08

decision_id: `D08`
title: Minute precision only at the engine boundary

decision:
Canonical input time precision for v1 is minute level only. Seconds are not accepted at the engine boundary.

rationale:
This decision was made to keep the boundary strict and simple, avoid unnecessary precision complexity, and prevent hidden normalization behavior inside the engine.

rejected_alternatives:

- accepting second precision directly
- accepting multiple time precision modes in v1
- silently normalizing seconds inside the engine

affected_documents:

- Document 04
- Document 09

implementation_risk_prevented:

- unstable input precision expectations
- hidden coercion behavior
- unnecessary complexity in the initial implementation

folded_back_status:
folded_back

Decision D09

decision_id: `D09`
title: Old Gate 1 anchor assumption rejected

decision:
The previous assumption that Gate 1 Line 1 begins at `313.25°` is rejected. The adopted canonical gate sequence and anchor basis are defined in the final mapping canon.

rationale:
The earlier anchor assumption conflicted with the annual Human Design transit cycle and with the adopted gate sequence logic. Keeping it would have corrupted downstream gate assignment.

rejected_alternatives:

- retaining the old Gate 1 anchor
- continuing with an unverified anchor out of convenience

affected_documents:

- Document 05
- Document 06
- Document 09

implementation_risk_prevented:

- incorrect gate mapping

- incorrect Gene Keys identifiers
- incorrect structural derivation from wrong gates

folded_back_status:
folded_back

Decision D10

decision_id: `D10`
title: Canonical gate sequence and anchor adopted explicitly

decision:
The Human Design gate order list and its adopted anchor basis were explicitly pinned in the mapping canon, rather than left as a provisional or inferred sequence.

rationale:
This decision was made because gate mapping is foundational. A reconstructed but undeclared sequence would invite reinterpretation by the implementer. Once adopted into canon, it becomes law for v1.

rejected_alternatives:

- leaving the sequence as a working candidate
- permitting external Human Design sources to override the adopted list
- treating the gate sequence as provisional during implementation

affected_documents:

- Document 05
- Document 06
- Document 09

implementation_risk_prevented:

- alternative sequence substitution
- gate assignment inconsistency
- downstream errors in Gene Keys, channels, centers, definition, Type, and Authority

folded_back_status:
folded_back

Decision D11

decision_id: `D11`
title: Human Design structural rules must be explicit, not imported by convention

decision:
The final canon explicitly adopts deterministic rules for:

- channels
- centers
- definition
- Type
- Authority
- Incarnation Cross structural encoding

These are not left to “standard Human Design rules” without formal specification.

rationale:
The master calculation specification named several Human Design structural outputs without fully formalizing all derivation logic. The final canon had to close that gap explicitly.

rejected_alternatives:

- relying on “standard Human Design rules” without explicit canon wording
- allowing the implementer to infer BodyGraph mechanics from general domain knowledge

affected_documents:

- Document 05
- Document 06
- Document 09

implementation_risk_prevented:

- implementer interpretation
- divergence between platforms and canon
- hidden disputes about how structural BodyGraph logic should work

folded_back_status:
folded_back

Decision D12

decision_id: D12
title: Incarnation Cross is structural only in v1

decision:
For v1, Incarnation Cross is output as structural encoding only:

- Personality Sun gate and line
- Personality Earth gate and line
- Design Sun gate and line

- Design Earth gate and line

A human readable cross name is not required unless a naming table is explicitly pinned later.

rationale:
This decision avoids fake precision. The structure can be pinned deterministically even when a naming layer is not yet formally canonized.

rejected_alternatives:

- inventing or assuming a naming table without pinned authority
- outputting only partial cross information
- mixing structural output and interpretive naming layers

affected_documents:

- Document 05
- Document 06
- Document 07
- Document 09

implementation_risk_prevented:

- ambiguous naming behavior
- false confidence in non pinned naming sources
- mixing deterministic output with presentation concerns

folded_back_status:
folded_back

Decision D13

decision_id: D13
title: One pinned canon version only for v1

decision:
Trinity Engine v1 supports one pinned canon version only.

rationale:
This decision avoids premature complexity, compatibility burden, and version selection ambiguity in the first implementation.

rejected_alternatives:

- multi canon support in v1
- runtime selectable canon versions
- compatibility shims in v1

affected_documents:

- Document 01
- Document 08
- Document 09

implementation_risk_prevented:

- bloated test matrix
- compatibility ambiguity
- version selection logic introduced before the canon is stable

folded_back_status:
folded_back

Decision D14

decision_id: `D14`
title: Swiss Ephemeris version pin

decision:
Swiss Ephemeris version `2.10.03` is the authoritative astronomical source version for Trinity Engine v1.

rationale:
This decision removes source drift risk and aligns implementation, testing, and review on one pinned astronomical source version.

rejected_alternatives:

- leaving the version unspecified
- allowing implementer choice
- using latest available version

affected_documents:

- Document 03
- Document 08
- Document 09

implementation_risk_prevented:

- result differences caused by source version drift
- unverifiable implementation differences
- arguments over whether mismatches are code defects or ephemeris differences

folded_back_status:
folded_back

Decision D15

decision_id: `D15`
title: Hashes excluded from canonical output in v1

decision:
Normalized input hashes and output hashes are not included in canonical engine output for v1.

rationale:
This decision keeps v1 focused on deterministic calculation and essential version metadata without adding unnecessary implementation burden to the canonical contract.

rejected_alternatives:

- including hashes immediately in canonical output
- treating operational hash logging as part of the engine contract

affected_documents:

- Document 01
- Document 07
- Document 09

implementation_risk_prevented:

- unnecessary complexity in the canonical output contract
- avoidable implementation disagreement about hash construction rules
- mixing operational tooling concerns into the engine response

folded_back_status:
folded_back

Decision D16

decision_id: `D16`
title: Output precision and datetime format are explicitly pinned

decision:
Pinned output formatting for v1 is:

- longitude values are numeric and rounded to 6 decimal places
- `design_time_utc` uses ISO 8601 UTC format with trailing `Z`
- `design_time_utc` uses whole second precision

rationale:
This decision was made to eliminate output ambiguity and make fixtures, regression testing, and implementation review exact rather than interpretive.

rejected_alternatives:

- leaving precision unspecified
- allowing variable datetime formatting
- using millisecond precision by default

affected_documents:

- Document 07
- Document 09

implementation_risk_prevented:

- fixture mismatch caused by formatting drift
- inconsistent output serialization
- disputes over whether two outputs are “equivalent enough”

folded_back_status:
folded_back

Decision D17

decision_id: `D17`
title: Channel ordering and identifier hygiene are explicit

decision:
Canonical identifier hygiene and ordering rules were hardened explicitly, including:

- ascending lexicographic `channel_id` order in output
- use of `ego` as the canonical center identifier
- rejection of alias drift such as `will` as a separate engine identifier

rationale:
This decision was made because identifier drift creates fake bugs, output mismatches, and unnecessary disputes that have nothing to do with calculation correctness.

rejected_alternatives:

- leaving output ordering soft
- allowing center aliases in canonical output
- relying on presentation labels inside engine identifiers

affected_documents:

- Document 06
- Document 07
- Document 09

implementation_risk_prevented:

- inconsistent outputs across runs or implementations
- silent identifier drift
- avoidable fixture failures

folded_back_status:
folded_back

Decision D18

decision_id: D18
title: Golden Test Pack is part of the implementation requirement, not founder side prepopulation

decision:
Document 10 defines the required Golden Test Pack structure, coverage, and exactness rules. The actual fixture population and execution are expected as part of the implementation and review deliverables.

rationale:
This decision clarifies responsibility. The canon should define the required test pack. The implementer should build and execute the fixtures as part of delivering a compliant engine.

rejected_alternatives:

- implying that the founders must prepopulate all fixtures before implementation begins
- treating the test pack as optional
- allowing fixture creation to remain informal or ad hoc

affected_documents:

- Document 10
- Document 11
- Document 12

implementation_risk_prevented:

- unclear ownership of test obligations
- weak acceptance enforcement
- handwaving around what must actually be tested

folded_back_status:
folded_back

Decision D19

decision_id: D19
title: Full transparency package shared with implementation engineer

decision:
The full package, including Document 00 and Document 09, is shared with the implementation engineer for transparency.

rationale:
This decision was made to eliminate hidden assumption layers and make the full reasoning, ambiguity handling, and control posture visible during review.

rejected_alternatives:

- sharing only the core canon documents and keeping control and ambiguity documents internal
- relying on oral explanation of decision history

affected_documents:

- Document 00
- Document 09
- Document 12

implementation_risk_prevented:

- backchannel canon behavior
- untraceable disagreements about why certain rules exist
- reintroduction of rejected assumptions during review

folded_back_status:
folded_back

Decision D20

decision_id: D20
title: Pinned timezone database release required for deterministic UTC conversion
decision:
In addition to the pinned astronomical source version, Trinity Engine v1 must use one explicitly pinned timezone database release as the authoritative timezone reference for canonical UTC conversion.

rationale:
Historical timezone and daylight saving rules can change across timezone database

releases. Leaving the timezone database family unpinned would allow cross environment differences in UTC conversion, which would then propagate into astronomical calculations and downstream domain outputs.

`rejected_alternatives:`

- treating “IANA timezone data” as sufficiently specific without a pinned release
- allowing environment default timezone data to govern canonical conversion
- accepting runtime specific timezone data variation as harmless

`affected_documents:`

- Document 03
- Document 04
- Document 08
- Document 09

`implementation_risk_prevented:`

- cross environment UTC conversion drift
- hidden dependency drift outside the pinned source stack
- output mismatches caused by differing historical timezone definitions

`folded_back_status:`
folded_back

Decision D21

`decision_id:` D21
`title:` House interval ownership is explicitly cusp inclusive and next cusp exclusive
`decision:`
For astrology house placement in Trinity Engine v1, each house interval begins at its own cusp inclusive and ends at the next cusp exclusive. A longitude exactly equal to a house cusp belongs to the house beginning at that cusp. Wraparound between House 12 and House 1 follows the same rule.

`rationale:`
House placement cannot be deterministic if cusp ownership is left to library behavior, implied astrology convention, or implementer judgment. An explicit interval ownership rule is required to prevent ambiguity at house boundaries.

`rejected_alternatives:`

- leaving house cusp ownership to external library defaults
- treating house ownership as implicitly understood from astrology convention
- using a dual inclusive interpretation that risks double assignment

`affected_documents:`

- Document 05
- Document 06
- Document 09

`implementation_risk_prevented:`

- boundary ambiguity at house cusps
- cross implementation drift in house assignment
- disputes over whether boundary cases are canon defects or implementation choices

`folded_back_status:`
folded_back

Decision D22

`decision_id:` D22
`title:` Design time final timestamp uses lower bound selection under sub second bracket stop condition
`decision:`
If the Design time search stops because the remaining interval width is less than 1 second, the canonical Design time is the lower bound of the final search interval. The emitted `design_time_utc` value uses whole second precision by truncation to that lower bound.

`rationale:`
Bisection stop conditions alone do not determine which canonical timestamp must be emitted when the remaining bracket is narrower than one second. An explicit selection and emission rule is required to prevent hidden rounding discretion.

`rejected_alternatives:`

- leaving final timestamp selection implied by the search process
- using nearest second rounding without explicit canon adoption
- using midpoint selection
- using upper bound selection without formal canon wording

`affected_documents:`

- Document 05
- Document 07
- Document 09
- Document 10

`implementation_risk_prevented:`

- cross implementation drift in `design_time_utc`
- fixture instability at sub second convergence boundaries

- hidden serializer or rounding behavior affecting canonical output

`folded_back_status:`
folded_back

Decision D23

`decision_id:` D23
`title:` Canonical error comparison is based on error type and error code, not free text message wording
`decision:`
For Trinity Engine v1, canonical error comparison uses `error_type` and `error_code` as authoritative fields. The `message` field remains required developer facing text, but it is not authoritative for fixture exactness unless a later canon revision pins a formal message catalog.

`rationale:`
Free text message exactness creates brittle fixtures and fake disagreement unless the project is willing to own a formal message catalog. Stable machine level comparison is better grounded in typed canonical fields.

`rejected_alternatives:`

- treating free text error messages as exact canonical truth without a catalog
- relying only on coarse error types without specific error codes
- allowing implementers to choose their own error granularity without canon guidance

`affected_documents:`

- Document 07
- Document 09
- Document 10

`implementation_risk_prevented:`

- brittle fixture failures caused by wording differences
- false disputes over implementation correctness
- ambiguous classification of similar but distinct failure conditions

`folded_back_status:`
folded_back

Decision D24

`decision_id:` D24
`title:` Timezone inputs are restricted to canonical IANA Area slash Location identifiers
`decision:`
For Trinity Engine v1, accepted timezone identifiers are restricted to canonical IANA `Area/Location` names. Abbreviations, aliases, and link names are not accepted unless explicitly introduced by later canon revision.

`rationale:`
A strict timezone identifier boundary avoids hidden normalization, alias ambiguity, and cross environment interpretation differences. The engine boundary should remain narrow and deterministic rather than permissive.

`rejected_alternatives:`

- accepting timezone abbreviations such as CET or EST
- accepting alias forms such as US/Eastern by default
- silently normalizing non canonical timezone identifiers inside the engine

`affected_documents:`

- Document 04
- Document 07
- Document 09

`implementation_risk_prevented:`

- timezone parsing ambiguity
- hidden normalization behavior
- disagreements over whether non canonical identifiers should be accepted

`folded_back_status:`
folded_back

Decision D25

`decision_id:` D25
`title:` Success response comparison is semantic JSON equality, not literal byte for byte serialization
`decision:`
Canonical success response comparison for Trinity Engine v1 is based on semantic JSON equality against the output contract. Exact scalar values, canonical field names, and canon pinned ordered arrays must match, but literal byte for byte object serialization is not required unless a later canon revision explicitly pins serializer behavior and object member ordering.

`rationale:`
Literal byte for byte matching claims false precision when serializer behavior and object member ordering are not themselves canonized. Semantic JSON equality preserves strictness where it matters without inventing requirements the canon does not actually pin.

`rejected_alternatives:`

- implying byte for byte equality without pinning serializer behavior
- weakening comparison to loose logical similarity
- allowing unordered collections where canonical order is explicitly required

`affected_documents:`

- Document 07
- Document 09
- Document 10

`implementation_risk_prevented:`

- fake strictness in acceptance review
- unnecessary implementation disagreements caused by serializer details
- confusion about what exactness actually means in the canon

`folded_back_status:`
folded_back

Decision D26

`decision_id:` D26
`title:` First golden fixture freeze requires approval review beyond initial engine generation
`decision:`
The first approved golden fixture set for Trinity Engine v1 must not be frozen solely because it was generated by a canon compliant engine. Candidate fixtures must also undergo review against the governing canon, including independent review of ambiguity prone boundary areas, before being marked approved.

`rationale:`
A fixture generated by the same implementation later judged against it is not automatically trustworthy as a frozen oracle. Initial fixture freeze requires governance review to avoid circular self validation.

`rejected_alternatives:`

- freezing the first fixture set immediately after initial engine generation
- relying on implementation output alone as sufficient oracle creation
- treating fixture approval as implicit rather than explicit

`affected_documents:`

- Document 09
- Document 10
- Document 12

`implementation_risk_prevented:`

- circular oracle construction
- weak acceptance governance
- future disputes over whether fixtures prove canon compliance or merely implementation self consistency

`folded_back_status:`
folded_back

---

5. Dispute Handling Guidance

If Pascal identifies an issue during review or implementation, the issue should be classified as one of the following:

- **intentional canon choice**
  The behavior is unusual or strict, but it is explicitly intended and folded back into the governing canon.
- **canon defect**
  The canon is incomplete, inconsistent, or wrong and requires formal correction.
- **implementation defect**
  The canon is clear, but the implementation does not match it.
- **fixture defect**
  The expected test result is wrong relative to the governing canon.

This document helps support that distinction, but the actual fix must always be made in the governing canon documents where applicable.

---

6. Final Use Rule

This document should be read together with:

- Document 00 for status and dependency tracking
- Document 09 for ambiguity and fold back tracking
- Documents 01 to 08 and 11 for governing implementation rules
- Document 10 for golden test pack requirements

This document improves transparency and collaboration, but it does not replace the canon itself.